

Katuwang — Finalized System Design

A web-based peer tutoring management platform for Taysan Resettlement Integrated School (Grades 7–12)

Finalized architecture • database schema (ERD) • process & data-flow diagrams • UI/UX refinements • four-part design justification

The problem Katuwang solves

- Context: Taysan Resettlement Integrated School (TRIS), Legazpi City - a public secondary school in a resettlement community, under classroom overcrowding and split shifts.
- Problem: students reach senior high still lacking basic reading / math skills; teachers are overloaded and often teaching out-of-field; the school has no formal system to organize peer academic support.
- Katuwang connects student tutors (peer volunteers) with student learners so tutoring is organized, tracked, and moderated instead of ad hoc.
- Everything is voluntary and free - no payments or monetization anywhere.
- Privacy mandate (RA 10173): double-blind anonymity - learners and tutors never see each other's real name, email, or contact info.

END USERS

Student Learner - Student Tutor - Admin
(Grades 7-12)

SIX MODULES

User Mgmt - Session Mgmt - Tutor Matching -
Assessment - Chatbot - Analytics

SCOPE NOTE

3 roles only - the thesis' 4th 'Teacher
Moderator' role was cut; duties fold into Admin.

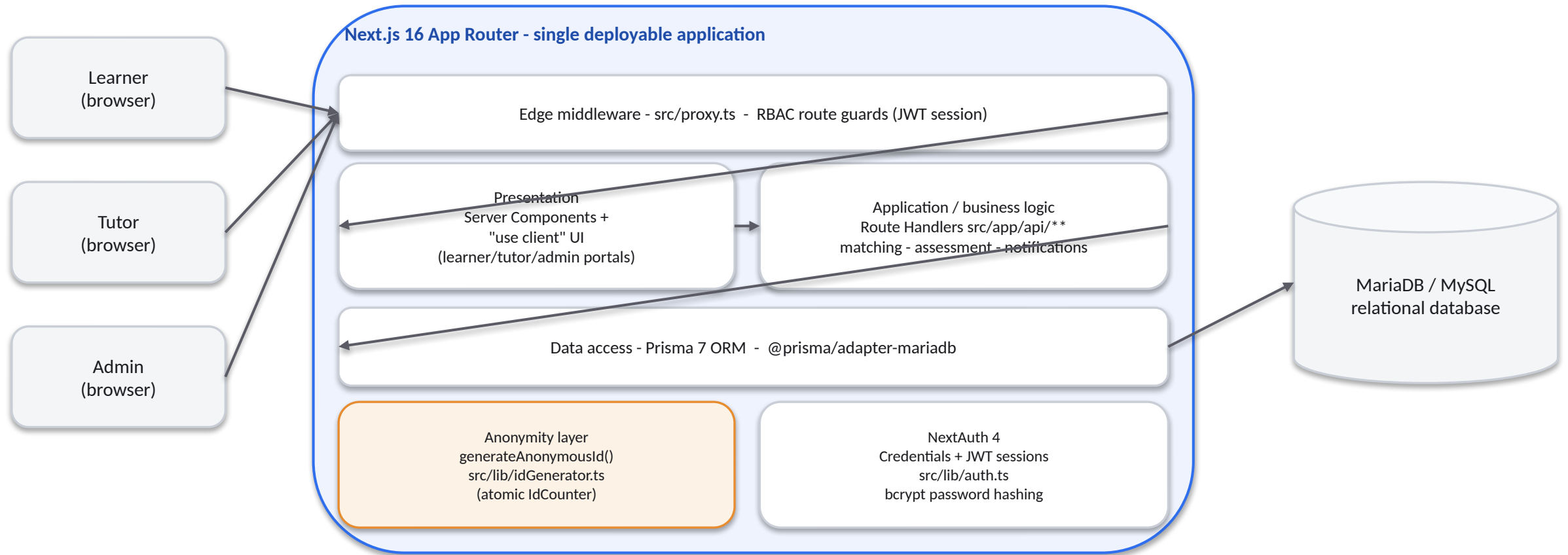
What our Capstone 1 panel asked for

Recorded from the panel recommendation sheet. Each is resolved later in this deck with a specific design change and a named justification.

#	Panel recommendation	Artifact affected	Resolved on
R1	The data model had no way to record a tutor's qualification to teach a specific topic - only a single "is a tutor" flag, with no record of which subject/topic was approved, by whom, or when. Model the tutor certification lifecycle explicitly, enforceable and auditable per topic.	Database schema (ERD), DFD	Slides 6-8, 10-11
R2	[Architecture recommendation from your panel sheet - e.g. component left unspecified, or a technology questioned for scale. Replace this row.]	Architecture	Slides 4-5
R3	[UI/UX recommendation from your panel sheet - e.g. a missing empty / error state a user would actually hit. Replace this row.]	UI/UX mockups	Slide 9

R1 is the one panel recommendation documented in our repo answers (docs/to-submit/...Katuwang-answered.docx, Item 12). BEFORE PRESENTING: pull the team's actual Capstone 1 panel recommendation sheet and replace R2 / R3 with the real recommendations - or, if the panel gave only one, delete those rows and say so.

Pattern: modular monolith (Next.js full-stack)



Every request: browser -> edge RBAC guard -> Server Component / Route Handler -> Prisma -> MariaDB. Peer-facing responses pass through the anonymity layer.

Why modular monolith - and each component's job

- Chosen over microservices: at capstone scale a 5-person team cannot operate independent services; the modules are loosely related, not independently scaled or deployed.
- Chosen over a plain layered app on a separate backend: Next.js unifies presentation + API in one codebase and one deploy, which the team can actually build and test in the timeline.
- Internal module separation still gives us layered discipline (presentation / logic / data) without a distributed system.
- Scale target: one school - hundreds of active students, not millions.

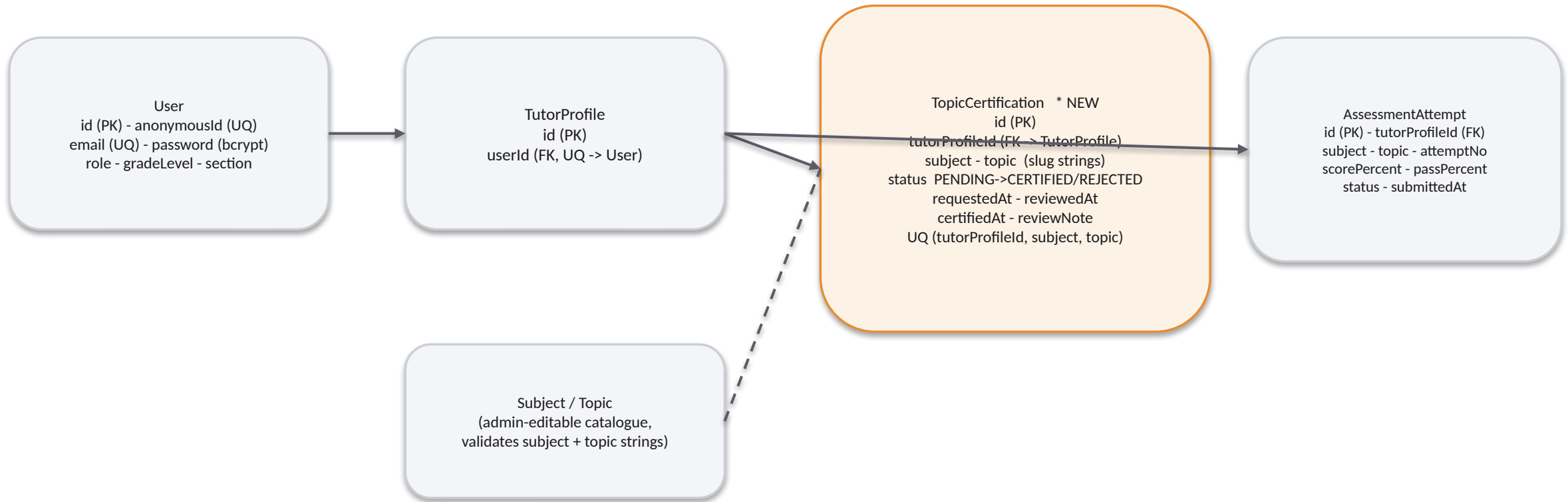
Component	One-sentence responsibility
Edge middleware (proxy.ts)	Rejects any request whose JWT role may not reach that route.
Server Components / client UI	Renders each role's portal and collects user input.
Route Handlers (api/**)	Validate (Zod), enforce RBAC, run business rules, return JSON.
matching.ts	Ranks classes for a learner by weighted score.
Prisma + adapter	Only path to the database; owns transactions.
idGenerator.ts	Issues collision-free anonymous IDs atomically.
NextAuth (auth.ts)	Authenticates credentials, mints and verifies JWT sessions.

Technology stack

Layer	Technology	Role
Framework	Next.js 16 (App Router)	Monolithic full-stack app (RSC + client components)
Language	TypeScript 5 (strict)	Static typing across models, validation, APIs
UI & styling	Tailwind CSS 4 + DaisyUI 5	OKLCH semantic theme, accessible components
Icons	Lucide React	Action / status / navigation icons
Authentication	NextAuth.js 4	Credentials provider, JWT session strategy
Database & ORM	MariaDB / MySQL + Prisma 7	Schema modeling, migrations, @prisma/adapters-mariadb
Validation	Zod 4	Runtime request-body validation, discriminated unions
Testing	Vitest 4	Unit + API route integration tests

When asked about tools, refer to the type - diagramming tool, database/ERD modeling tool, UI/UX design tool - per the module's instruction.

ERD - the certification lifecycle (R1)



Crow's-foot reading: a User has zero-or-one TutorProfile; a TutorProfile has zero-or-many TopicCertifications; each TopicCertification belongs to exactly one TutorProfile. FK TopicCertification.tutorProfileId -> TutorProfile ON DELETE CASCADE.

Data dictionary - TopicCertification

Attribute	Type	Constraint	Description
id	String (cuid)	PK	Surrogate key
tutorProfileId	String	FK -> TutorProfile, NOT NULL, ON DELETE CASCADE	Owning tutor
subject	String	NOT NULL, validated vs Subject.slug	Subject slug, e.g. "MATH"
topic	String	NOT NULL, validated vs Topic.name	Topic within the subject
status	Enum	NOT NULL, default PENDING	PENDING -> CERTIFIED REJECTED
requestedAt	DateTime	NOT NULL, default now()	When the tutor requested it
reviewedAt	DateTime	NULL until an admin decides	Audit: decision timestamp
certifiedAt	DateTime	NULL unless CERTIFIED	Audit: when eligibility began
reviewNote	Text	NULL	Admin feedback, mainly for a rejection
(composite)	-	UNIQUE (tutorProfileId, subject, topic)	One row per tutor-topic pair

Six schema-refinement checks applied: entity completeness (entity was implied but never drawn) - attribute definition (typed status + audit fields; password marked bcrypt on User) - PK/FK (all named) - cardinality & participation (stated in sentence form) - 3NF (no partial/transitive dependencies) - referential integrity (CASCADE on the tutor FK).

New states surfaced during refinement

[SCREENSHOT]
Tutor - "Not yet certified
for this topic" gate

blocks opening a class for a
topic with no CERTIFIED row;
CTA -> take the qualifying quiz

[SCREENSHOT]
Tutor - certification
PENDING state

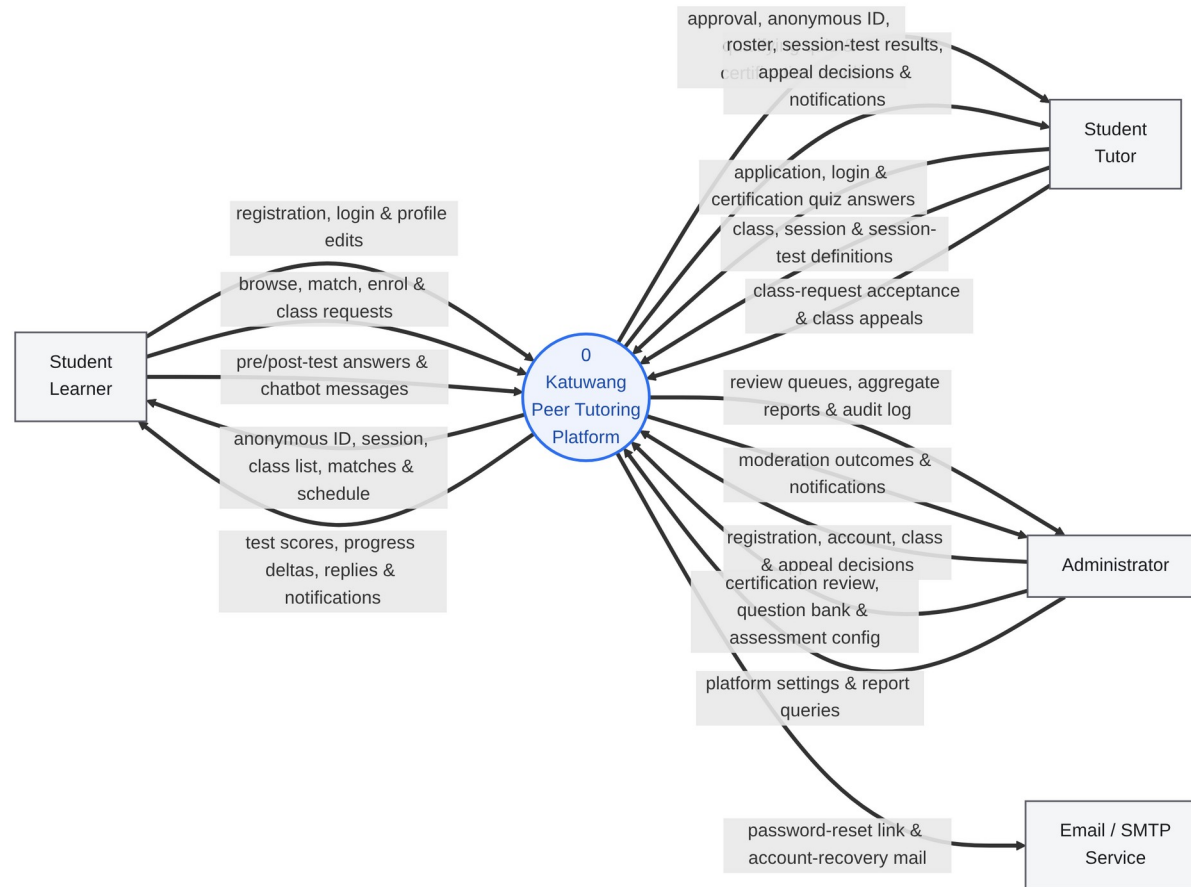
request submitted, awaiting
admin review; status chip +
requestedAt shown

[SCREENSHOT]
Learner - class browser
empty state

no class matches the filter ->
CTA to post a class request
instead of a dead end

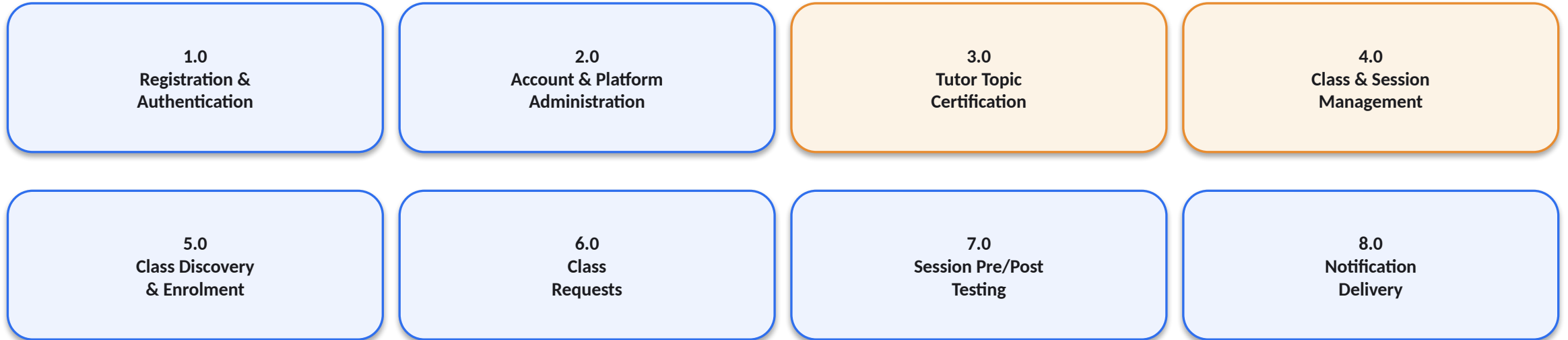
- Refinement = rough wireframe -> validated high-fidelity mockup: every state a user actually hits, including empty / error / confirmation states.
- Heuristics satisfied: visibility of system status (PENDING chip), error prevention & user control (no dead ends - always a next action), match between system and real world (a tutor can't teach what they aren't certified for).
- Capture the three screenshots from the running redesigned app (pnpm dev + seed accounts) and drop them into assets/ui/ before presenting.

Context diagram (Level 0)



The whole platform as one process with four external entities. Every learner-tutor flow carries only the anonymous ID (STU-#### / TUT-####) - the RA 10173 double-blind boundary. Email/SMTP is the one outside service (password reset + account recovery).

Level 1 - eight processes



12 data stores: DS1 Users & Tutor Profiles - DS2 Subjects & Topics - DS3 Topic Certifications - DS4 Question Bank & Attempts - DS5 Tutor Classes & Sessions - DS6 Class Enrolments - DS7 Class Requests - DS8 Session Tests & Attempts - DS9 Class Appeals - DS10 Notifications - DS11 Audit Log - DS12 Platform & Assessment Settings

Detailed walkthrough - subprocess 4.0: 3.0 writes a PENDING row to DS3 -> tutor passes the qualifying quiz (questions from DS4, origin = BANK) -> admin certifies -> DS3 row = CERTIFIED -> 4.0 reads DS3 as the per-topic eligibility gate before a class can open. Traceability verified: every DS maps to a Prisma entity (docs/erd.md); every process maps to an architecture component (slide 4); every change traces to a panel rec (slide 3).

Most significant change - functionality & security

Change justified: modelling the tutor certification lifecycle explicitly (TopicCertification entity + per-topic eligibility gate).

FUNCTIONALITY

- Directly satisfies the requirement the panel flagged as unmet: teaching eligibility is now recorded, enforced, and auditable per (tutor, subject, topic).
- Unblocks the rule "a tutor may open a class for a topic only with a CERTIFIED row" - impossible with the old single boolean flag.
- Audit trail (requestedAt / reviewedAt / certifiedAt / reviewNote) supports admin review and dispute handling.

SECURITY

- Certification is admin-gated - a tutor cannot self-certify; status transitions happen only on an authenticated ADMIN route.
- Passwords stored as bcrypt hashes (User.password); sessions are signed JWTs.
- Double-blind anonymity preserved: peer-facing responses expose only id / anonymousId, never name or email; anonymous IDs minted atomically via generateAnonymousId() so concurrent registrations can't collide.

Most significant change - scalability & problem alignment

SCALABILITY

- UNIQUE (tutorProfileId, subject, topic) doubles as the lookup index - the eligibility check is a single indexed row read on every class-creation attempt.
- Foreign keys indexed; list endpoints paginated.
- Realistic target: one school - hundreds of active tutors/learners, tens of certifications per tutor. No sharding or service split needed; the modular monolith absorbs this load on one deploy.

PROBLEM ALIGNMENT

- The original problem: TRIS has no formal system to organise and moderate peer academic support.
- A per-topic certification gate is exactly the quality-control mechanism a formal system needs - it ensures a peer teaching a topic has demonstrably passed that topic.
- It adds no complexity unrelated to that problem: no new role, no external service, no feature outside the six scoped modules.

Recommendation -> change -> artifact -> justification

Panel rec.	Design change	Artifact updated	Justification headline
R1 - no per-topic tutor qualification in the data model	New TopicCertification entity; per-topic eligibility gate on class creation	ERD + data dictionary; DFD process 3.0 + gate DS3->4.0; architecture (assessment module)	Functionality: makes eligibility enforceable/auditable; aligns with the "formal, moderated peer support" problem
R2 - [architecture rec]	[change]	Architecture diagram	[headline]
R3 - [UI/UX rec]	[new empty/error state]	UI/UX mockups	[heuristic + headline]

- Deliverables: finalized architecture diagram, ERD + data dictionary, Context + Level 1 DFDs, refined UI/UX mockups, four-part justification.
- Every artifact is internally consistent - DFD stores <-> ERD entities <-> architecture components - and every change is traceable to a panel recommendation.
- Fill R2 / R3 from the real panel sheet before submission.

Questions

Anticipated - every member ready to field one:

- Why a modular monolith and not microservices or a separate backend?
- Where exactly is the double-blind anonymity boundary enforced, and what stops a leak?
- Why an enum status on TopicCertification instead of a separate review-history entity?
- Which functional-requirement numbers does the certification change satisfy?
- Is there any DFD data store with no matching ERD entity? (No - traceability was checked.)
- How does a tutor move from PENDING to CERTIFIED? (Pass that topic's qualifying quiz, then admin review.)

Tools used: a diagramming tool, a database/ERD modeling tool, a UI/UX design tool.

Speaker map, timing & artifact index

Section	Slides	Time	Speaker
Intro & problem recap	1-2	~1 min	Member 1
Panel recommendations	3	1-2 min	Member 1
Finalized architecture	4-6	~2 min	Member 2
Finalized schema (ERD)	7-8	~2 min	Member 3
UI/UX refinements	9	~1 min	Member 4
Process & data-flow diagrams	10-11	~2 min	Member 4
Design justification + summary	12-14	2-3 min	Member 5
Q&A	15	2-3 min	All

- Full 30-entity ERD: docs/erd.md (Mermaid).
- DFD source + full flow list: assets/DFD.md, assets/dfd-level0.mmd, assets/dfd-level1.mmd.
- Schema source of truth: prisma/schema.prisma.
- Written module answers: docs/to-submit/...Katuwang-answered.docx.
- Scope divergences: docs/reference/decisions.md.
- Screenshots to add: assets/ui/ (pnpm dev + seed accounts).

Core content ~ 11-13 min, leaving 2-3 min for Q&A - inside the 10-15 min limit. Rehearse against a timer; confirm every member speaks (Rubric Criterion 4).

DFD Level 1 - process catalogue & data stores

#	Process	Portal / route group
1.0	Registration & Authentication	/register, /api/register, /api/auth/*
2.0	Account & Platform Administration	/admin/users, /admin/registrations, /admin/assessment/*, /admin/settings, /admin/reports
3.0	Tutor Topic Certification	/tutor/assessments, /api/tutor/topic-certifications, /admin/assessment/certifications
4.0	Class & Session Management	/tutor/classes, /api/tutor/classes/*
5.0	Class Discovery & Enrolment	/learner/classes, /learner/match, /api/classes, /api/learner/match
6.0	Class Requests	/learner/requests, /tutor/requests, /admin/topic-requests
7.0	Session Pre/Post Testing	/tutor/.../sessions/[id]/test, /learner/.../test/[kind], /admin/session-tests
8.0	Notification Delivery	/api/notifications, /{role}/notifications

Data store	Prisma model(s)
DS1 Users & Tutor Profiles	User, TutorProfile, IdCounter, PasswordResetToken
DS2 Subjects & Topics	Subject, Topic
DS3 Topic Certifications	TopicCertification
DS4 Question Bank & Attempts	AssessmentQuestion/Option/Attempt/Item, QuestionRequest
DS5 Tutor Classes & Sessions	TutorClass, ClassTopic, ClassSession
DS6 Class Enrolments	ClassEnrollment
DS7 Class Requests	TopicRequest(+Topic,+Slot)
DS8 Session Tests & Attempts	SessionTest(+Question,+Attempt,+Item)
DS9 Class Appeals	ClassAppeal
DS10 Notifications	Notification (+ ChatbotMiss)
DS11 Audit Log	AuditLog
DS12 Platform & Assessment Settings	PlatformSetting

DFD Level 1 - full data-flow diagram

